

ML-POWERED KUBERNETES MONITORING

K8s AI-Ops

From metric scrape to AI alert: how Prometheus, Grafana, Loki, KEDA and a custom MLOps layer connect, why every tool was chosen over its alternatives, and how each piece works internally - explained step by step for interview deep-dives.

Architecture & Interview Guide · October 2026 ·
k8s-ai-ops v1.0

Table of Contents

1. Executive Summary - The System in One Page	3
2. The Problem - Why Static Monitoring Is Not Enough	4
2.1 The three failures of threshold-based monitoring	4
2.2 Where MLOps enters	4
3. Architecture at a Glance	5
4. Kubernetes - The Foundation That Makes Everything Measurable	6
4.1 What it is and why it is the right substrate	6
4.2 How metrics actually leave a cluster - three exporters, three jobs	6
4.3 The ServiceMonitor pattern	7
5. Prometheus - The Metrics Brain	7
5.1 What it is	7
5.2 How the pull model works, and why it beats push	7
5.3 PromQL - the one language every later stage speaks	7
5.4 Why Prometheus and not the alternatives	8
6. Grafana - The Single Pane of Glass	8
6.1 How provisioning works here (no clicking)	8
6.2 What the AI Ops dashboard shows	8
6.3 Why Grafana and not Kibana	9
7. Loki + Promtail - Logs Without a Search Cluster	9
7.1 How the pipeline works	9
7.2 The design trade-off to articulate	9
7.3 Why Loki and not the EFK stack	10
8. Alertmanager - One Alert Bus for Rules and ML Alike	10
8.1 What it does	10
8.2 The design twist worth explaining in interviews	10
9. KEDA - Event-Driven Autoscaling	11
9.1 What it is and how it works internally	11
9.2 The two ScaledObjects in this project	11
9.3 Why KEDA and not plain HPA	11
10. Redis and the PVC - The Bus and the Filing Cabinet	12
10.1 Redis as the work queue	12

10.2 The PVC as artifact store	12
11. The Custom MLOps Layer - Collector, Trainer, Analyzer	12
11.1 Feature engineering - the vocabulary every stage shares	12
11.2 The collector - a two-line contract with big consequences	13
11.3 The trainer - two models and a self-labeling trick	13
11.4 The analyzer - scoring, alerting, explaining	15
12. All the Dots Connected - One Window's Journey	16
13. Scaling and Resilience Properties	17
14. Anatomy of an Alert and an RCA Report	17
15. Deployment Walkthrough - From Zero to Alerting	18
16. Proving It Live - Two Scripted Incidents	18
16.1 Error storm (scripts/smoke-test.sh, incident 1)	18
16.2 OOMKill (incident 2)	19
17. Design Decision Cheat Sheet - "Why X, Not Y"	19
18. Rapid-Fire Interview Q&A	20
19. Glossary - Terms to Use Precisely	22

1. Executive Summary - The System in One Page

This document explains **k8s-ai-ops**, a production-style MLOps platform deployed on Kubernetes that watches both **cluster health** (nodes, pods, volumes) and **application health** (request rate, errors, latency), **learns what normal looks like** for every workload, **predicts failures before they happen**, raises alerts through the same pipeline as classical alerts, and attaches **concrete debugging steps plus the relevant logs and Kubernetes events** to every alert. Scaling of the analysis layer is event-driven via KEDA.

The one-sentence pitch you can give an interviewer: "It is a self-learning monitoring layer on top of the standard Prometheus/Grafana/Loki stack - a collector turns live metrics into feature windows, a trainer learns per-workload baselines and a failure predictor without any manually labeled incidents, an analyzer scores every window and pushes alerts with root-cause context, and KEDA scales the whole inference layer by queue depth."

60 s feature window cadence (collector)	98th pct anomaly threshold calibration	15 min failure prediction horizon	6 h retraining cadence (CronJob)	1 - 6 analyzer replicas (KEDA range)
--	---	--	---	---

Eleven building blocks cooperate: Prometheus, Alertmanager, Grafana, Loki, Promtail, KEDA, Redis, a persistent volume, and three custom Python services (collector, trainer, analyzer) plus a deliberately breakable demo application. Sections 4-10 explain each tool, what it does, how it works internally, why it is necessary, and why this tool was chosen over its alternatives. Section 11 dissects the three custom services. Section 12 connects every dot in one numbered end-to-end walkthrough. Sections 17-19 are the interview cheat sheets: a design decision table, rapid-fire Q&A, and a glossary.

Document part	What it answers	Sections
Foundation	What problem does ML-on-monitoring solve, and what does the architecture look like?	2 - 3
Tools	What does each tool do, how does it work, why this one and not the alternative?	4 - 10
The MLOps layer	How do the collector, trainer and analyzer actually work, line of code up?	11
End-to-end flow	How does one CPU spike become an alert with debugging steps and logs?	12 - 14
Operations	How is it deployed, and how are incidents demonstrated live?	15 - 16
Interview prep	Decision cheat sheet, rapid-fire Q&A, glossary.	17 - 19

Table 1 - How to read this document.

2. The Problem - Why Static Monitoring Is Not Enough

2.1 The three failures of threshold-based monitoring

Classical monitoring compares each metric against one static threshold chosen by a human. That fails in three predictable ways, and all three motivate this project:

- **Baselines differ per workload.** A batch job at 90% CPU is healthy; a payment API at 90% CPU is an incident. One global threshold cannot serve both, and writing per-workload thresholds by hand does not scale beyond a handful of services.
- **Static thresholds only catch known-unknowns.** They fire on the failure modes someone already thought of. Memory leaks that creep slowly, intermittent network drops, or a slow dependency degradation do not cross any single threshold - they cross a *pattern*.
- **Alert fatigue.** Thresholds tuned high miss real events; tuned low they page constantly. The industry result is well known: on-call engineers ignore noisy alert sources.

ML attacks all three: a learned per-workload baseline replaces hand-tuned thresholds, a supervised model recognizes *precursor patterns* before the hard failure, and a calibrated threshold plus deduplication keeps the signal-to-noise ratio high.

2.2 Where MLOps enters

A one-off anomaly detection notebook is a demo; a **system** needs the full MLOps loop. MLOps here means, concretely: an automated data pipeline (the collector writes feature history), automated training with versioned artifacts (the trainer CronJob), automated serving with zero-downtime model swaps (the analyzer hot-reload), automated retraining on a schedule, and monitoring *of the model itself* (model age, rows trained, queue depth - all exported as Prometheus metrics and shown on the Grafana dashboard). Every one of those five pieces exists in this codebase and is examinable in an interview.

Key phrase for interviews: "The hard part is not the model - it is the loop. Features, labels, training, serving, retraining and observability-of-the-model must all be automated, or the model rots within weeks."

3. Architecture at a Glance

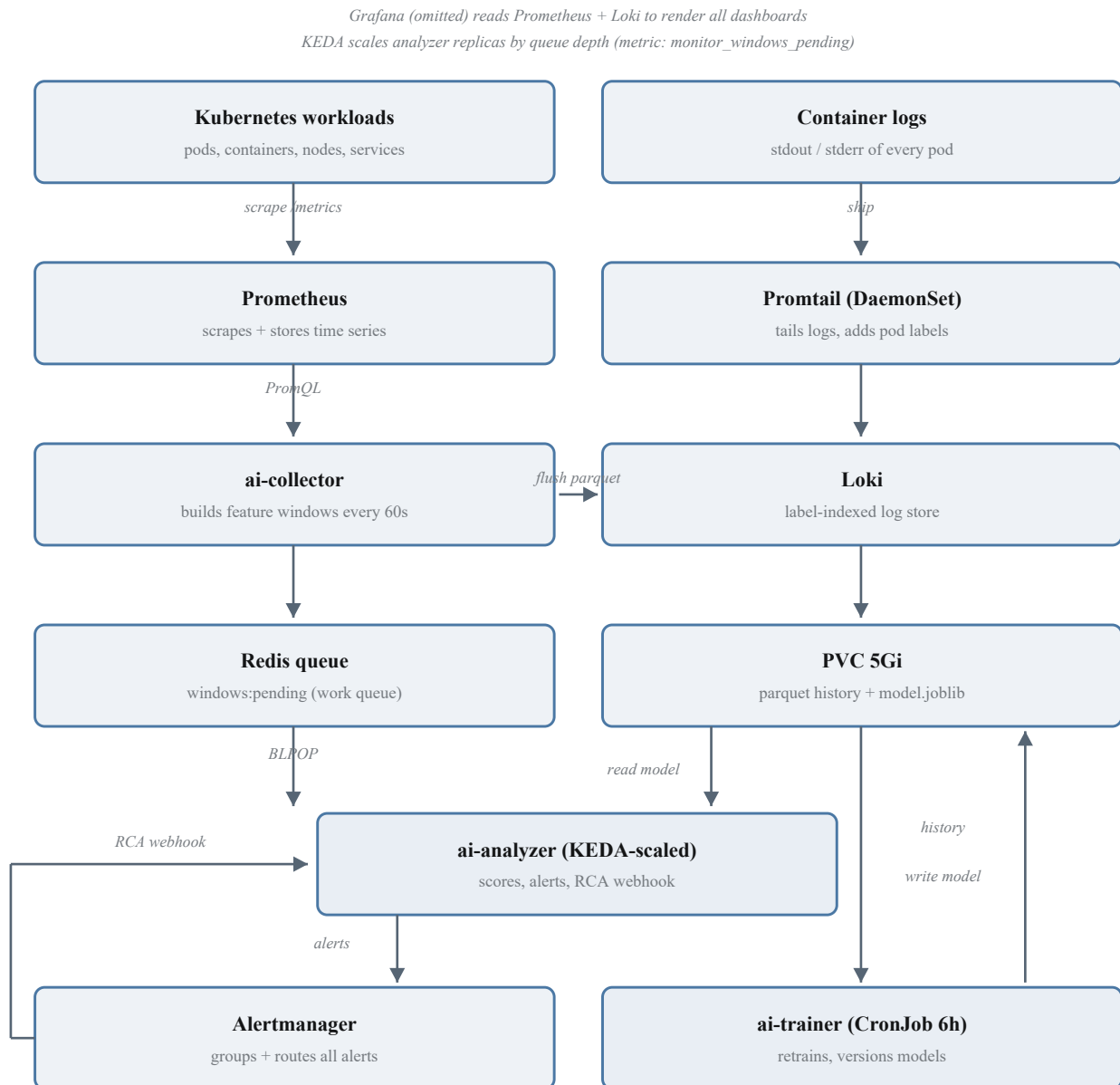


Figure 1 - Component map. Solid arrows are data flows; the loop on the bottom-left to bottom-right is the RCA enrichment round-trip.

Component	Runs as	Namespace	One-line responsibility
Prometheus	StatefulSet (kube-prometheus-stack)	monitoring	Scrapes and stores all metrics; answers PromQL
Alertmanager	Deployment (same chart)	monitoring	Groups, deduplicates and routes alerts
Grafana	Deployment (same chart)	monitoring	Dashboards over metrics and logs
Loki	Deployment (helm chart)	monitoring	Stores logs, indexed by labels only
Promtail	DaemonSet (one per node)	monitoring	Ships container logs to Loki with k8s labels
KEDA operator	Deployment	keda	Event-driven autoscaling from Prometheus queries
ai-collector	Deployment (1 replica)	ai-ops	PromQL -> feature windows -> Redis + parquet

Component	Runs as	Namespace	One-line responsibility
ai-trainer	Job + CronJob	ai-ops	Trains anomaly + failure models on the PVC
ai-analyzer	Deployment (KEDA 1-6)	ai-ops	Scores windows, pushes alerts, enriches RCA
Redis	Deployment	ai-ops	Work queue, history ring, dedup keys
demo-app + loadgen	Deployment x2	demo	Breakable app producing realistic telemetry

Table 2 - Component inventory (three namespaces: monitoring, ai-ops, demo).

Three namespaces separate concerns: **monitoring** is off-the-shelf infrastructure, **ai-ops** is the custom MLOps layer, and **demo** is a sacrificial workload used to generate real incidents. This separation maps to Kubernetes RBAC: the analyzer's ServiceAccount can only *read* pods, events and nodes - it cannot modify the cluster.

4. Kubernetes - The Foundation That Makes Everything Measurable

4.1 What it is and why it is the right substrate

Kubernetes schedules containers across a fleet of machines and continuously reconciles actual state toward declared state. It matters to this project for one central reason: **every meaningful signal is already a first-class object**. Pod restarts, OOM kills, readiness failures, node memory pressure, PVC fill levels - the platform itself knows about all of them and exposes them. An ML monitor built on Kubernetes can therefore learn from the system's own truth, instead of bolting agents onto VMs.

4.2 How metrics actually leave a cluster - three exporters, three jobs

Source	What it is	What it exposes	Why it exists
kubelet / cAdvisor	Agent on every node	container_cpu_usage_seconds_total, container_memory_working_set_bytes	Live per-container resource usage
node-exporter	DaemonSet (one pod per node)	node_cpu_seconds_total, node_memory_MemAvailable_bytes, filesystem metrics	Host-level health the kubelet does not expose
kube-state-metrics	Deployment (cluster-level)	kube_pod_status_ready, kube_pod_container_status_restarts_total, kube_node_status_condition, volume stats	Kubernetes <i>object</i> state - restarts, conditions, limits - not usage

Table 3 - The three metric sources. A classic interview trap: kube-state-metrics reports object state (e.g. restart counts) while cAdvisor reports usage - you need both.

4.3 The ServiceMonitor pattern

Prometheus here runs with the **Prometheus Operator** (part of kube-prometheus-stack). Instead of hand-editing a scrape config, monitoring targets are declared as a **ServiceMonitor** custom resource - a small YAML that says "scrape port http of services with this label". The operator watches these CRDs and rewrites Prometheus's configuration. Every app in this project (demo-app, collector, analyzer) ships its own ServiceMonitor, so onboarding an app into monitoring is one declarative file - no config drift, GitOps-friendly.

Why Kubernetes at all (interview sound-bite): the cluster already encodes health as objects and events. We do not reinvent detection of "pod restarted" - we learn the *patterns that precede it* and attach the cluster's own evidence to the alert.

5. Prometheus - The Metrics Brain

5.1 What it is

Prometheus is a time-series database plus a scraper plus a query language. Every few seconds it **pulls** an HTTP endpoint (`/metrics`) on each target and stores the numeric samples with their label sets. It also evaluates **PromQL** expressions - the language used everywhere in this project, from Grafana panels to the collector's feature queries to KEDA's scaling triggers.

5.2 How the pull model works, and why it beats push

In pull-based monitoring the server initiates: Prometheus discovers targets (from Kubernetes API service discovery), fetches each target every scrape interval, and records success. A failed scrape produces the synthetic metric **up == 0** - so "is the target alive" is itself a queryable time series. In a push world a crashed exporter leaves the server holding a stale last value that looks healthy; in a pull world the absence of a successful scrape is loud and immediate. Pull also means the monitoring server decides the cadence and can be re-scraped at higher rate while debugging - no client reconfiguration.

5.3 PromQL - the one language every later stage speaks

```
Example - the exact shape of queries inside src/common/features.py
# CPU of each pod as a fraction of its limit (2-minute rate)
sum by (namespace, pod) (rate(container_cpu_usage_seconds_total[2m]))
/ max by (namespace, pod) (kube_pod_container_resource_limits{resource="cpu"})

# 99.5th-percentile request latency per service
histogram_quantile(0.95, sum by (le, namespace, service)
(rate(http_request_duration_seconds_bucket[2m])))
```

Four PromQL primitives cover the whole project: **rate()** turns counters into per-second velocity, **sum by ()** aggregates and keeps grouping labels, **histogram_quantile()** computes latency percentiles from bucketed histograms, and vector matching (`/`) joins two series by label - which is how usage is divided by limits to

produce the ratios the models eat.

5.4 Why Prometheus and not the alternatives

Alternative	Why not, for this system
Datadog / New Relic / CloudWatch (SaaS)	Per-host/per-metric pricing explodes with pod count; metrics leave the cluster; the ML layer would query a paid remote API on a 60s loop; vendor lock-in. Prometheus is free, runs in-cluster, and is the CNCF-graduated de-facto Kubernetes standard.
InfluxDB	A fine time-series DB but push-oriented and app-agnostic: no native Kubernetes service discovery, no kube-state-metrics equivalent, no ecosystem of exporters. We would rebuild scraping ourselves.
Elastic metrics / OpenTSDB	Heavier operational footprint for the same job; weaker Kubernetes-first tooling.
Push gateway only	PushGateways are for short batch jobs. For long-running pods, pull + up-metric is strictly more truthful.

Table 4 - Prometheus versus alternatives.

Two design consequences matter downstream: the collector, KEDA and Grafana all speak the same PromQL dialect (one mental model), and alert rules are evaluated by the same engine that stores the data - no export layer in between to go stale.

6. Grafana - The Single Pane of Glass

Grafana is a visualization layer that talks to many backends through a datasource plugin model. In this stack one Grafana instance holds a Prometheus datasource (metrics) and a Loki datasource (logs), so a responder pivots from a CPU graph to the matching log lines without changing tools - the classic **metrics-to-logs drill-down** that makes incidents diagnosable.

6.1 How provisioning works here (no clicking)

The dashboard is a JSON file in the repository (dashboards/ai-ops-overview.json). The deploy script wraps it in a Kubernetes ConfigMap labelled **grafana_dashboard=1**; a Grafana sidecar container watches all namespaces for ConfigMaps with that label and loads them automatically. Datasources are provisioned the same way from Helm values. Consequence to say out loud in interviews: *dashboards are code* - versioned, reviewable, reproducible on any cluster in minutes.

6.2 What the AI Ops dashboard shows

Panel	Query essence	Question it answers
Model age (s)	monitor_model_age_seconds	Is the ML layer alive and freshly trained?
Windows pending	monitor_windows_pending	Is the analysis queue draining (KEDA health)?

Panel	Query essence	Question it answers
AI alerts sent (1h)	increase(monitor_alerts_sent_total[1h])	Is the AI layer noisy or quiet?
Anomaly score top-10	topk(10, monitor_anomaly_score)	Which entities deviate most right now?
Failure risk top-10	topk(10, monitor_failure_risk)	What is predicted to break next?
Node CPU/mem, restarts, 5xx, p95	standard PromQL	Classical cluster + app health context

Table 5 - Dashboard panels. The first three monitor the monitor itself.

6.3 Why Grafana and not Kibana

Kibana is excellent but log-first (Elasticsearch-centric); mixing Prometheus panels into it is second-class. Grafana is metrics-first with first-class Loki support, supports provisioning-as-code natively, and is what the wider ecosystem (kube-prometheus-stack) assumes. Choosing Kibana would buy better full-text log search - which Loki's design deliberately trades away (section 7) - at the cost of fragmenting the UI story.

7. Loki + Promtail - Logs Without a Search Cluster

7.1 How the pipeline works

Promtail runs as a DaemonSet - one pod per node - mounts /var/log and every container's log file, and ships lines to Loki. Before shipping it attaches **Kubernetes labels** it discovers from the file path and the API: namespace, pod, container. **Loki** stores log streams grouped by label set and builds an index *only over those labels*; the raw lines are compressed into chunks in object/file storage.

7.2 The design trade-off to articulate

Elasticsearch builds a full inverted index over every word of every log line - that is why search is instant and why the cluster is huge and expensive. Loki indexes only labels and then **grep-scans chunks** at query time. The result: roughly an order of magnitude cheaper storage and far simpler operations, at the price of slower ad-hoc full-text search. This system's queries are always label-scoped - *show me error lines for pod X in namespace Y* - which is exactly the shape Loki is optimized for. When the RCA webhook needs logs for an alert, it already knows the labels (they come from the alert itself).

```
RCA log retrieval - label selector plus regex filter, LogQL
# The exact query the analyzer's RCA module runs (src/analyzer/enrich.py)
query_range:
  query: '{namespace="demo", pod="demo-app-abc"} |~ "(?i)error|exception|fatal|panic"'
  limit: 15, direction: backward, window: last 30 minutes
```

7.3 Why Loki and not the EFK stack

Alternative	Why not, for this system
Elasticsearch + Fluentd/Fluent Bit (EFK)	Full-text index over all cluster logs needs multiple data nodes, JVM tuning and heavy storage; we need per-pod error retrieval, not log analytics.
CloudWatch / Stackdriver	Same SaaS objections as section 5.4; also the RCA webhook would need cloud credentials inside the cluster.
Fluentd instead of Promtail	Promtail discovers Kubernetes metadata natively and shares the Grafana ecosystem; Fluentd adds a Ruby-based pipeline for no gain here.

Table 6 - Logging alternatives.

8. Alertmanager - One Alert Bus for Rules and ML Alike

8.1 What it does

Alertmanager receives alert JSON on its `/api/v2/alerts` endpoint and applies four powers: **deduplication** (identical label sets collapse into one), **grouping** (group_by alertname+namespace - one notification per group, not per pod), **silencing and inhibition** (mute during maintenance; a critical alert can suppress its own warnings), and **routing** (a receiver tree decides who gets what). It also handles repeat_interval and resolve notifications.

8.2 The design twist worth explaining in interviews

AI alerts flow **into** Alertmanager through its API, and **every** alert - Prometheus rules and AI alerts alike - is routed **back out** to the analyzer's `/webhook/alerts` receiver. That single loop buys three things: one enrichment path (RCA code written once, used by both alert origins), one notification destination (Slack is configured in exactly one place), and one source of truth for what is currently firing. The alternative - analyzer posts straight to Slack - would fork routing logic and lose grouping.

Configured behavior	Value in this repo	Why
Route receiver	ai-rca webhook (send_resolved: true)	All alerts get RCA enrichment, resolved ones included
group_by	alertname, namespace	50 crashing pods produce one grouped notification, not fifty
group_wait / group_interval	30 s / 5 m	Batch the first burst, then incremental updates
repeat_interval	4 h	Persistent problems re-notify without spamming

Table 7 - Alertmanager settings (k8s/monitoring/kube-prometheus-stack-values.yaml).

9. KEDA - Event-Driven Autoscaling

9.1 What it is and how it works internally

KEDA (Kubernetes Event-driven Autoscaling) is an operator that extends the built-in Horizontal Pod Autoscaler. You declare a **ScaledObject** CRD pointing at a Deployment plus a **scaler**; KEDA creates and manages a real HPA behind the scenes. Its control loop polls the event source every **pollingInterval** (15 s here), computes **value / threshold**, and feeds that as the HPA's external metric - so replicas track queue depth the same way stock HPA tracks CPU. With **minReplicaCount: 1** the analyzer webhook always stays warm (a scale-to-zero HTTP receiver would drop webhooks during cold starts); **cooldownPeriod: 120** prevents flapping after bursts.

9.2 The two ScaledObjects in this project

```
k8s/keda/scaledobjects.yaml (abridged)
# ai-analyzer: scale by analysis backlog
triggers: [{type: prometheus,
  serverAddress: http://kube-prometheus-stack-prometheus.monitoring.svc:9090,
  query: max(monitor_windows_pending) or vector(0), threshold: "25"}]
# -> 1..6 replicas; every 25 queued windows = +1 scoring worker set
# demo-app: scale by real traffic
query: sum(rate(http_requests_total{namespace="demo"}[2m])) or vector(0)
threshold: "40" # -> 1..8 replicas
```

9.3 Why KEDA and not plain HPA

Aspect	Plain HPA	KEDA
Metric sources	Resource metrics (CPU/mem) or custom-metrics API - which requires running a metrics adapter per source	60+ scalers out of the box, including Prometheus: just a query + threshold in YAML
Queue-depth scaling	Not possible without building an adapter	Native, one ScaledObject
Scale to zero	No	Yes (disabled here on purpose for the webhook)
Under the hood	It is the scaling engine	KEDA manages an HPA for you - so HPA knowledge still applies

Table 8 - KEDA versus plain HPA.

The elegant part: the scaling signal (`monitor_windows_pending`) is itself a metric the collector exports - the autoscaler watches a gauge that measures the system's own backlog. Backpressure-aware ML serving, in one sentence.

10. Redis and the PVC - The Bus and the Filing Cabinet

10.1 Redis as the work queue

The collector pushes each feature window with **LPUSH** onto a list (**windows:pending**, capped at 5000 with **LTRIM** - a bounded queue, the oldest windows are dropped first under overload). Analyzer workers block on **BLPOP** - each window is consumed by exactly one worker, which is the simplest correct work-queue semantics. Redis also holds three side structures: a per-entity **history ring** (last 64 windows, for lag features at scoring time), **dedup keys** (alerted: with 15-minute TTL), and the **results/rca lists** the /results and /rca API endpoints read.

Alternative	Why not, for this system
Kafka	Throughput here is tiny (one window per pod per minute - hundreds of messages/min). Kafka's partitions, consumer groups and broker operations buy durability and fan-out we do not need; it would be the most over-engineered line in the architecture.
RabbitMQ	Solid queue, but AMQP broker management for a workload Redis lists already handle; Redis was needed anyway for history + dedup, so the queue rides the same dependency.
No queue (score inline in collector)	Couples data collection to ML serving latency; a model crash would stall telemetry; no horizontal scaling story. The queue is what makes KEDA scaling possible.

Table 9 - Queue alternatives. The honest trade-off: Redis lists are at-least-once and unpersisted by default - acceptable because windows are transient and recomputable.

10.2 The PVC as artifact store

A 5Gi PersistentVolumeClaim (ReadWriteOnce) mounted into collector, trainer and analyzer holds **parquet feature history** (one file per day) and the **versioned model artifacts** (model.joblib + model.meta.json). Parquet is chosen over CSV/JSON because it is columnar (trainers read only feature columns), compressed, schema-typed and fast for pandas. The stated production upgrade path: swap RWO for a ReadWriteMany storage class or push artifacts to S3 - the code reads a path, so the change is configuration, not architecture.

11. The Custom MLOps Layer - Collector, Trainer, Analyzer

11.1 Feature engineering - the vocabulary every stage shares

src/common/features.py defines 16 PromQL queries grouped into four entity types. This file is the contract between the metric world and the model world; all three services import it.

Entity	Features per window	Design intent
pod (9)	cpu_ratio, mem_ratio (usage / limit), restarts_15m, oom_recent, not_ready, net_err_10m, net_drop_10m	Ratios normalize across pod sizes; hard signals give the rules layer truth
svc (3)	rps, http_5xx_rate, latency_p95	The three pillars of service health (RED method)
node (3)	node_cpu_used, node_mem_used, pressure_signals	Saturation and kernel-level pressure
pvc (1)	pvc_used_ratio	Disk-full is the classic slow-burn outage
lags (5)	cpu/mem rolling mean+std over last 6 windows, restarts_sum	Trend features: is this entity ramping toward failure?

Table 10 - The feature set (ALL_COLS in features.py). One JSON row per entity per 60 s.

11.2 The collector - a two-line contract with big consequences

Every COLLECT_INTERVAL seconds the collector runs `build_frames()`, converts the result to one JSON row per entity, and fans each row out to: the Redis queue (consumption), the entity's history ring (lag features), and an in-memory buffer flushed to daily parquet files every 20 cycles (training). It also exports **monitor_windows_pending** - the gauge KEDA scales on. Note the separation of concerns: the collector knows nothing about models; the trainer knows nothing about Prometheus.

11.3 The trainer - two models and a self-labeling trick

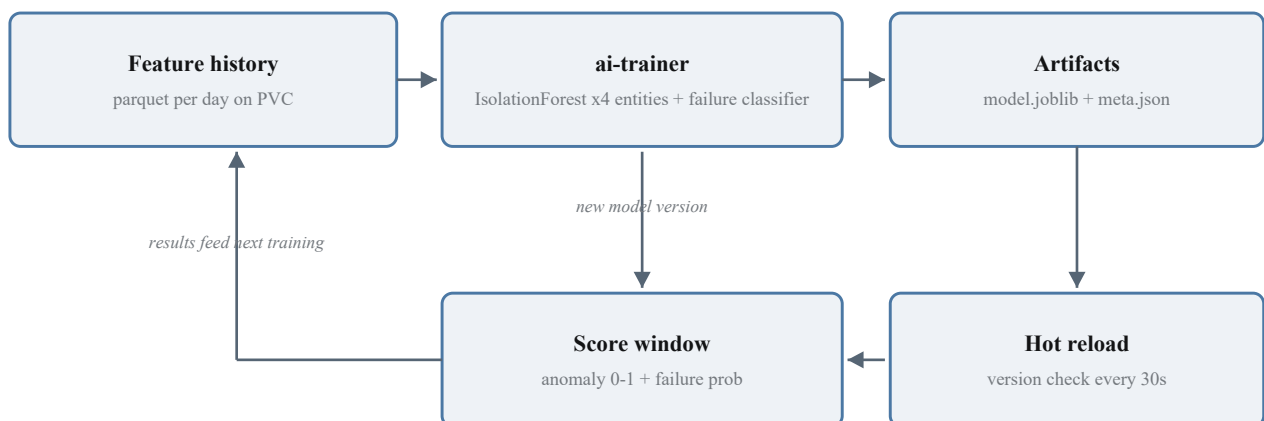


Figure 2 - The training loop: history in, versioned artifacts out, hot-reloaded at serving; accumulating results feed the next retrain.

Model 1: IsolationForest anomaly detection (one per entity type)

An IsolationForest grows an ensemble of random trees that split on a random feature at a random value. Outliers - being few and different - are **isolated in fewer splits**, so their average path length is short. The score is derived from that path length: no distance metric, no density estimate, near-linear training. The pipeline standardizes features first (StandardScaler), trains on all history for that entity type, then **calibrates a threshold** at the 98th percentile of the training scores - "alert on the 2% most unusual windows" is an interpretable,

tunable contract. The trainer also stores the per-feature **median and IQR** of normal data; at scoring time the features that deviate most (in IQR units) become the human-readable *top factors* - the why behind the score.

Candidate	Verdict for this workload	Reason
IsolationForest (chosen)	Yes	Fast on many entities, no distance metric, native multivariate, easy threshold calibration + explanation via feature deviations
One-Class SVM	No	Kernel training is superlinear in samples; sensitive to nu/gamma tuning; poor fit for continuous retraining
Local Outlier Factor	No	Distance-based, transductive by default (scoring new points is awkward); struggles when feature scales shift across namespaces
Autoencoder	No	Needs far more data and GPU-grade serving for 14 features; reconstruction-error thresholds are opaque; overkill
Per-metric 3-sigma	Partial	Kept as the hard-signal rules layer, but univariate - misses cross-feature patterns (high CPU + rising memory + restlessness)

Table 11 - Anomaly model selection.

Model 2: HistGradientBoostingClassifier - predicting failure 15 minutes out

The second model answers: *will this pod hit a restart, OOM kill or not-ready state within the next 15 minutes?* The interview-worthy trick is **where labels come from**: `build_supervised()` walks each pod's history chronologically and marks a training row positive if a failure signal occurs in the following `PREDICTION_HORIZON_MIN` minutes. Supervision is mined from the telemetry itself - **self-supervised labeling** - so no human incident tickets, no manual tagging, and every new day of history automatically produces new training data. The classifier consumes pod features plus the lag features (rolling mean/std), which is what lets it fire on a *ramp* rather than on the failure itself. The probability threshold is tuned by sweeping 0.05-0.95 and keeping the F1-best cut.

Candidate	Verdict	Reason
HistGradientBoosting (chosen)	Yes	Histogram-binned gradient boosting: state of the art on small/medium tabular data, native NaN handling, seconds to train, no GPU
Neural network (MLP/LSTM)	No	Tabular data of this size - GBMs consistently match or beat NNs; training/serving cost and tuning burden not justified
Logistic regression	No	Cannot capture non-linear interactions (memory-ratio plus rising-trend plus restart-history) without manual feature crosses
Prophet / time-series forecasters	No	Forecast a metric value, not a discrete failure event; per-metric again, multivariate patterns lost

Table 12 - Failure-model selection.

Cold start, versioning, retraining

- **Day 0:** no history exists, so the trainer bootstraps on synthetic-but-plausible ranges - anomaly alerts and hard signals work immediately; the supervised classifier activates only after ~300 rows / 25 real positives (roughly a day, faster with smoke tests).
- **Versioning:** artifacts are written atomically (tmp file + os.replace) as model.joblib + model.meta.json with a timestamp version - a reader never observes a half-written model.
- **Retraining:** a Kubernetes CronJob runs the trainer every 6 hours; a bootstrap Job covers first deploy; manual retraining is one kubectl create job command.

11.4 The analyzer - scoring, alerting, explaining

The analyzer is a FastAPI service with worker threads that BLPOP windows off the queue. For each window it: hot-reloads the model if model.meta.json shows a new version (checked every 30 s); computes the anomaly score normalized to 0-1 (min-max over the training score range) and, for pods, the failure probability from the classifier; derives top factors from IQR deviations; exports the two per-entity gauges **monitor_anomaly_score** and **monitor_failure_risk**; stores the result (visible at /results); and decides whether to alert.

Alert layer	Trigger	Alert name	Severity
Prometheus rules	Known failure modes: CrashLoop, OOMKilled, node pressure, PVC fill, 5xx rate, p95 latency, target down	PodCrashLoopBackOff, ContainerOOMKilled, ...	warning / critical
ML hard signals	Restart, OOM or not-ready observed right now - alerts even with no model loaded	AIHardSignal	critical
ML anomaly	IsolationForest score above the calibrated 98th-percentile threshold	AIAnomalyDetected	warning
ML prediction	Classifier probability above its F1-tuned threshold (e.g. 0.35)	AIPredictedFailure	critical

Table 13 - Four alert layers. Rules catch the known; ML catches the unknown and the not-yet-happened.

Alerts are **deduplicated per entity** (Redis key with 15-minute TTL) and carry an auto-resolve time (30 min). Each alert's annotations contain the summary, the scores, the top factors, and **debugging** - the first output of the knowledge base.

The debugging knowledge base and the RCA webhook

src/analyzer/suggestions.py maps anomaly signatures to concrete steps - real kubectl commands, not platitudes: an OOM flag yields "describe the pod, compare working set vs limit, check for a leak in Grafana, check rollout history"; a CPU anomaly yields "check throttling metrics, scale out, correlate with the last deploy"; a full PVC yields "expand or clean up, check log retention". Because Alertmanager routes *all* alerts back to /webhook/alerts, every alert is then enriched: the RCA module queries Loki for the pod's recent error lines, lists the pod's Kubernetes events via the official Python client (read-only ServiceAccount), assembles the report, stores it (at /rca) and forwards a formatted message to Slack when a webhook secret is configured. The result looks like: *alert name, entity, scores, last five events, last five error logs, six debugging steps* - a responder starts 80% done.

12. All the Dots Connected - One Window's Journey

Follow a single CPU-ramp incident through every component, in order. This is the narrative to reproduce on a whiteboard.

#	Where	What happens	The dot it connects
1	kubelet / cAdvisor, node-exporter, kube-state-metrics	Containers and nodes expose raw counters on /metrics	K8s -> Prometheus
2	Prometheus	Scrapes every 15-30 s via ServiceMonitor discovery; stores time series	Sources -> TSDB
3	Promtail -> Loki	Container logs shipped with namespace/pod labels	K8s logs -> Loki (for later RCA)
4	ai-collector	Every 60 s runs 16 PromQL queries, joins them per entity, computes ratios and lag features -> one JSON window per pod/svc/node/pvc	Prometheus -> features
5	ai-collector	LPUSH window to Redis queue + entity history ring; appends to parquet buffer; exports monitor_windows_pending	Features -> queue + history
6	KEDA	Polls the queue-depth metric every 15 s; if backlog > 25/replica, scales analyzer out (1-6); scale-in after 120 s quiet	Queue -> replicas
7	ai-analyzer workers	BLPOP each window; hot-reload model if version changed; IsolationForest score 0-1; classifier failure probability; top factors via IQR	Queue -> scores
8	ai-analyzer	Exports monitor_anomaly_score / monitor_failure_risk; stores result in results:recent	Scores -> Grafana + API
9	ai-analyzer	Anomaly, prediction or hard signal? Dedup key (15 min TTL) -> POST alert to Alertmanager API with debugging annotations	Scores -> alert bus
10	Alertmanager	Groups by alertname+namespace, applies repeat/silence rules, routes to the ai-rca webhook	Alert -> enrichment
11	ai-analyzer /webhook/alerts	Pulls pod error logs from Loki + pod events from k8s API, merges with suggestions KB -> RCA report; stores; forwards to Slack	Alert -> RCA + notification
12	ai-trainer CronJob (6 h)	Reads growing parquet history, retrains both models, writes next versioned artifacts; analyzer hot-reloads them	History -> better model (the MLOps loop)

Table 14 - The full journey: metric to model to alert to explanation.

Interview gold: the two closed loops. *Operational loop:* metrics -> features -> scores -> alerts -> RCA (seconds to minutes, synchronous-ish). *Learning loop:* features -> history -> trainer -> model -> better scores (hours, asynchronous). Naming these two loops crisply is what separates a memorized architecture from an understood one.

13. Scaling and Resilience Properties

The analyzer's workload is bursty by construction: one surge of anomalies creates a surge of windows. Scaling on **queue depth** is backpressure-aware - the system measures its own latency proxy and grows workers until the queue drains. Three properties to enumerate:

- **Queue as shock absorber.** A 10x anomaly burst is absorbed by the bounded list (5000) while KEDA adds replicas; nothing is lost because LTRIM drops the oldest windows only in pathological overload - and those windows are stale anyway.
- **Warm floor.** minReplicaCount 1 keeps the webhook always serving; scale-to-zero is deliberately not used for an HTTP receiver.
- **Failure isolation.** If the model fails to load, scoring degrades to rule-based hard signals and Prometheus rules keep firing; if Redis dies, the collector keeps scraping and logs the failure - monitoring of the cluster never dies with the ML layer.

14. Anatomy of an Alert and an RCA Report

A concrete alert payload (abridged) is worth memorizing:

```
Alert posted to Alertmanager /api/v2/alerts
labels:      {alertname: AIPredictedFailure, entity: pod/demo/demo-app-abc,
              namespace: demo, severity: critical, ai_monitor: "1"}
annotations: {summary: AIPredictedFailure on pod/demo/demo-app-abc,
              description: anomaly_score=0.912 failure_prob=0.994 model=20261005T234533;
              factors: mem_ratio=1.3 (3.8x IQR); cpu_ratio=1.4 (4.1x IQR),
              debugging: "- describe pod, compare working set vs limit\n- check leak
                          in Grafana\n- rollout history..."}
startsAt:    2026-10-05T23:50:00Z      endsAt: +30 min (auto-resolve)
```

RCA report field	Source	Example
alertname, severity, entity	alert labels	AIPredictedFailure / critical / pod/demo/demo-app-abc
description + top factors	analyzer scoring	mem_ratio 3.8x IQR above baseline
Kubernetes events	k8s API (read-only SA)	BackOff: restarting failed container
Error logs	Loki query_range	ValueError: synthetic failure
Debugging steps	suggestions.py KB	kubectl -n demo logs pod --previous --tail=100
Delivery	Redis rca:recent + Slack webhook	visible at /rca, formatted Slack block

Table 15 - RCA report contents (build_rca in src/analyzer/enrich.py).

15. Deployment Walkthrough - From Zero to Alerting

Step	Command (scripts/deploy-all.sh)	Why this order
1	helm repo add prometheus-community / grafana / kedacore	Pin chart sources
2	kubectl apply -f k8s/00-namespaces.yaml	monitoring / ai-ops / demo must exist before charts and workloads
3	helm upgrade --install kube-prometheus-stack -n monitoring -f values.yaml	Prometheus + Alertmanager + Grafana + operators; values wire the RCA webhook and Loki datasource
4	helm upgrade --install loki / promtail -n monitoring	Logs before apps, so early incidents already have context
5	helm upgrade --install keda -n keda	Autoscaler before the workloads it scales
6	docker build x2; kind load (local) or registry push (remote)	Images available to nodes before Deployments reference them
7	kubectl apply -f k8s/ml/	ConfigMap/Secret -> Redis -> PVC -> collector/trainer/analyzer (envFrom + volume refs resolve in order)
8	kubectl apply -f prometheus-rules.yaml, k8s/demo/, k8s/keda/	Alert rules, demo workload, autoscaling
9	ConfigMap from dashboard JSON + label grafana_dashboard=1	Sidecar auto-loads the dashboard
10	rollout status waits + job wait for bootstrap training	Block until the first model exists

Table 16 - Deployment order and its logic.

The same script auto-detects a kind cluster (images loaded into the node) versus a remote cluster (scripts/build-push.sh pushes to a registry and re-points all workloads). Port-forward script exposes Grafana (:3000), Prometheus (:9090), Alertmanager (:9093) and the analyzer API (:8088). One command teardown: scripts/clean.sh (helm uninstall + namespace deletes).

16. Proving It Live - Two Scripted Incidents

16.1 Error storm (scripts/smoke-test.sh, incident 1)

Time	Component	Observable reaction
t+0	kubectl set env ERROR_RATE=0.5 + rollout	demo-app pods restart with 50% 5xx injection
t+1 min	loadgen + Prometheus	http_requests_total{status=500} rate spikes; HTTPServerErrorRate rule counts toward firing

Time	Component	Observable reaction
t+2-6 min	collector + analyzer	svc windows show http_5xx_rate anomaly; AIAnomalyDetected or AIHardSignal dedup-posted
t+5 min	Alertmanager	Rule fires; grouped notification routed to RCA webhook
t+5 min	analyzer RCA	Loki returns error lines from demo pods; k8s events listed; Slack message with debugging steps
t+~8 min	KEDA (demo-app ScaledObject)	Error-path latency mix changes throughput; replicas scale 1-8 with RPS

Table 17 - Error storm timeline.

16.2 OOMKill (incident 2)

The demo app's /burn?mb=600 endpoint allocates 600 MiB inside a container limited to 256 MiB. The kernel OOM killer terminates the process; kubelet restarts it. Observables: kube_pod_container_status_last_terminated_reason{reason=OOMKilled} increments -> ContainerOOMKilled rule; the pod's next windows carry oom_recent=1 and restarts_15m >= 1 -> AIHardSignal (critical) with OOM-specific debugging steps; the event stream (via RCA) shows "OOMKilled, killing container"; and if several ramps occur across days, the classifier begins scoring the memory-ratio ramp *before* the kill - AIPredictedFailure, the system's headline demo.

17. Design Decision Cheat Sheet - "Why X, Not Y"

Decision	Chosen	Why not the alternative (one-liner)
Metrics backend	Prometheus (pull)	SaaS = cost + lock-in + data egress; InfluxDB = no k8s discovery/exporter ecosystem
Dashboards	Grafana	Kibana is log-first; Grafana unifies metrics + logs and provisions as code
Logs	Loki + Promtail	Elasticsearch full-text index is 10x the storage/ops for queries we never run
Queue	Redis lists	Kafka/RabbitMQ machinery unjustified at hundreds of msgs/min; Redis also serves history + dedup
Analyzer autoscaling	KEDA Prometheus scaler	Raw HPA cannot read an arbitrary PromQL metric without building an adapter
Anomaly model	IsolationForest per entity	OCSVM/LOF too slow/awkward to retrain continuously; autoencoder overkill; 3-sigma univariate
Failure model	HistGradientBoosting	Beats NNs on this tabular size; native NaN handling; trains in seconds on a CronJob
Labels	Self-supervised from history	No incident-ticket dependency; labels and features regenerate every day for free

Decision	Chosen	Why not the alternative (one-liner)
Serving	FastAPI + worker threads	Celery/microservices overhead unjustified for BLPOP + model.predict
Artifacts	joblib + meta.json on PVC	Atomic replace + version field; S3 is the production upgrade, not a redesign
Orchestration of training	k8s CronJob	Airflow/Preflight schedulers are a second platform to run for one job
RCA enrichment	Webhook loop via Alertmanager	Direct-to-Slack would fork routing and lose grouping/dedup

Table 18 - Every major choice with its rejection reason.

18. Rapid-Fire Interview Q&A

Q1. Walk me through the system in 60 seconds.

Prometheus scrapes Kubernetes and app metrics; Promtail ships logs to Loki. A collector turns metrics into per-entity feature windows every 60 s and pushes them to a Redis queue. An analyzer service - autoscaled by KEDA on queue depth - scores each window with an IsolationForest anomaly model and a gradient-boosted failure predictor, and pushes alerts into Alertmanager. Alertmanager routes all alerts back to the analyzer's webhook, which enriches them with Loki logs and Kubernetes events into an RCA report with debugging steps, then notifies Slack. A trainer CronJob retrains every 6 h on accumulating history; the analyzer hot-reloads new model versions.

Q2. Is the ML real or a wrapper?

Real and supervised where supervision is possible. Unsupervised IsolationForest per entity type with a calibrated 98th-percentile threshold handles unknown-unknowns; a supervised HistGradientBoosting classifier predicts failures 15 min ahead using labels mined automatically from history. Both retrain on a schedule with versioned artifacts and hot reload.

Q3. How does IsolationForest work, and why it over an autoencoder?

Random recursive splits isolate points; anomalies take short paths, and short average path length maps to a high score. It is near-linear, multivariate, and threshold-calibratable. An autoencoder needs much more data, GPU serving, and opaque reconstruction thresholds - unjustified for 14 features.

Q4. Where do labels come from without incident tickets?

Self-supervision: a training row is positive if the same pod shows a restart, OOM or not-ready signal within the next 15 minutes of history. The telemetry is its own ground truth.

Q5. How do you fight alert fatigue?

Four mechanisms: calibrated thresholds (2% quantile), per-entity dedup with 15-min TTL, Alertmanager grouping by alertname+namespace, and severity routing. Noise is measurable - monitor_alerts_sent_total is a dashboard panel.

Q6. What is MLOps here, concretely?

Automated feature pipeline (collector -> parquet), scheduled training with versioned atomic artifacts, zero-downtime serving with hot reload, autoscaled inference, and observability of the model itself (age, rows, queue depth as Prometheus metrics on a Grafana dashboard).

Q7. Why Redis and not Kafka?

Scale: one window per pod per minute. Kafka's durability and fan-out machinery is dead weight; Redis lists give exactly-one-consumer semantics with BLPOP and already host history rings and dedup keys. The trade-off - transient, at-least-once delivery - is acceptable for recomputable windows.

Q8. Why KEDA and not HPA?

HPA reads resource metrics or the custom-metrics API (which requires an adapter per source). KEDA ships a Prometheus scaler - a query and a threshold in YAML - and manages the HPA for me. It also offers scale-to-zero, which I deliberately disabled for the webhook.

Q9. Why Loki and not Elasticsearch?

Loki indexes only labels and grep-scans chunks - about an order of magnitude cheaper and simpler to operate. Our queries are always label-scoped (the alert carries the pod), which is Loki's exact sweet spot. We trade away ad-hoc full-text search we do not need.

Q10. What if the model is wrong?

Layers: deterministic Prometheus rules and hard signals keep firing regardless of model state; the ML threshold is calibrated, not absolute; alerts carry top factors so a human can judge quickly; and 6-hourly retraining adapts baselines. Honest answer: there is no feedback button yet - that is the next iteration.

Q11. What happens on day 0?

The trainer bootstraps from synthetic-plausible ranges, so anomaly + rule alerts work immediately. The supervised classifier activates after ~300 rows / 25 positives - about a day of history, faster with smoke tests. I can explain this trade-off because it is designed, not accidental.

Q12. How would you add an LLM to the RCA?

The plumbing already assembles structured context - scores, top factors, events, logs. An LLM node would narrate that context into a natural-language incident summary and suggest next actions; deterministic kubectl steps stay as guardrails.

Q13. How is it secured?

Namespace isolation, a read-only ServiceAccount (pods/events/nodes only) for event enrichment, Slack URL in a Secret, and no analytics component has write access to the cluster.

Q14. What are the current limitations?

RWO PVC ties writers to one node (fix: RWX class or S3); the suggestions KB is rule-based, not learned; no drift detector beyond model age; Redis is single-replica. Each has a stated upgrade path - which is the point.

Q15. How would you test it in an interview demo?

scripts/smoke-test.sh: set `ERROR_RATE=0.5` (error storm) and `/burn?mb=600` inside a 256 MiB limit (OOMKill). Alertmanager, /rca and Grafana show the full pipeline reacting within minutes.

19. Glossary - Terms to Use Precisely

Term	Precise meaning in this project
Prometheus	Pull-based time-series database + PromQL query engine; the metrics backbone
PromQL	Query language: <code>rate()</code> , <code>sum by ()</code> , <code>histogram_quantile()</code> , vector matching
Exporter	HTTP endpoint exposing counters/gauges (node-exporter, kube-state-metrics, apps)
kube-state-metrics	Converts Kubernetes object state (restarts, conditions, limits) into metrics
ServiceMonitor	CRD consumed by the Prometheus Operator to auto-configure scraping
Grafana	Visualization layer with Prometheus + Loki datasources; dashboards provisioned as ConfigMaps
Loki	Log backend indexing labels only; chunks grep-scanned at query time via LogQL
Promtail	DaemonSet shipping container logs with k8s metadata labels
Alertmanager	Alert bus: dedup, grouping, silencing, routing, repeat intervals; API v2
Webhook receiver	Alertmanager route delivering alert JSON to the analyzer's RCA endpoint
KEDA	Operator adding event-source autoscaling (Prometheus scaler here) on top of HPA
ScaledObject	KEDA CRD: target Deployment + scaler query + threshold + min/max replicas
HPA	Kubernetes Horizontal Pod Autoscaler - the engine KEDA manages under the hood
Redis list queue	LPUSH/LTRIM producer, BLPOP consumers - the feature-window work queue
Feature window	One JSON row: entity type, key, 9-14 feature values, timestamp
Lag features	Rolling mean/std of previous windows - the trend signal
IsolationForest	Ensemble anomaly detector; anomaly = short random-split path length
Isolation threshold	98th percentile of training scores; above it = anomaly
HistGradientBoosting	Histogram-binned gradient-boosted trees; the failure classifier
Self-supervised labels	Positives mined from future failure signals in history - no tickets
Hot reload	Analyzer re-reads model when meta.json version changes (30 s check)
PVC	PersistentVolumeClaim - 5Gi volume holding parquet history + model artifacts
OOMKilled	Kernel killed the container for exceeding its memory limit
CrashLoopBackOff	Pod repeatedly crashing; kubelet backs off restart cadence
RCA	Root-cause analysis report: scores + factors + events + logs + debug steps